

Memory Leak — Խորը ուղեցույց

C++ հարցազրույցի ամբողջական presentation

Այս ուղեցույցը մանրամասն բացատրում է memory leak-ը՝ ինչ է, երբ է լինում, ինչպես հայտնաբերել, ինչպես խուսափել, իրական օրինակներով, գործիքներով և հարցազրույցի հարցերով:

1. Ի՞նչ է Memory Leak-ը

Memory leak-ը լինում է, երբ ծրագիրը **heap-ում հիշողություն է վերցնում, բայց չի ազատում**, և այդ հիշողությունը այլևս հասցե չկա (pointer-ը կորել է): Այդ հիշողությունը մնում է «զբաղված» մինչև ծրագրի ավարտը:

```
void leak() {  
    int* p = new int(10);    // heap-ում 4 բայթ  
    // delete p; <- Մոռացված  
}  
// p-ն ոչնչանում է (stack), բայց heap-ի հիշողությունը մնում է
```

p-ն (stack pointer) ոչնչանում է ֆունկցիայից դուրս, բայց **new**-ով վերցրած heap հիշողությունը չի ազատվել, և այլևս ոչ մի pointer չի ցույց տալիս դրան -> **leak**:

2. Ինչո՞ւ է վտանգավոր

- Ծրագրի հիշողությունը աճում է աշխատանքի ընթացքում
- Վերջում՝ out of memory (bad_alloc), crash
- Երկարաժամկետ ծրագրեր (server, embedded) -> ավելի վտանգավոր
- Performance-ի անկում (swapping)

Կարճատև («one-time») ծրագրում մեկ leak-ը կրիտիկ չէ (OS-ն ազատում է վերջում), բայց server/loop-ում կուտակվում է ու սպանում ծրագիրը:

3. Ամենատարածված պատճառները (իրական օրինակներ)

3.1 delete-ի մոռացում

```
int* p = new int[100];  
// ... օգտագործում  
// delete[] p; <- մոռացան
```

3.2 Early return / exception

```
void f() {  
    int* p = new int(5);  
    if (someError) return; // LEAK -- delete չի կանչվում  
    delete p;  
}
```

3.3 Pointer-ի վերագրում (overwrite)

```
int* p = new int(1);  
p = new int(2); // առաջին new int(1)-ը կորավ -> LEAK  
delete p; // միայն երկրորդն է ազատվում
```

3.4 delete vs delete[] սխալ

```
int* arr = new int[100];  
delete arr; // Սխալ -- պետք է delete[] -> undefined / leak
```

3.5 Exception մեջտեղում

```
void f() {  
    int* p = new int(5);  
    mayThrow(); // եթե throw աներ -> delete չի հասնի -> LEAK  
    delete p;  
}
```

4. Ինչպես խուսափել

4.1 Ամեն new-ի համար delete

```
int* p = new int(5);  
// ...  
delete p;  
p = nullptr;           // dangling-ից խուսափելու
```

4.2 RAII / Smart pointers (ԼԱՎԱԳՈՒՅՆ ձև)

```
#include <memory>  
void f() {  
    auto p = std::make_unique<int>(5);  
    if (someError) return;    // ոչ մի leak -- unique_ptr-ն ինքն է ազատում  
}                             // scope-ից դուրս ավտոմատ delete
```

Modern C++-ում raw `new` / `delete` գրեթե չեն օգտագործվում: `unique_ptr` / `shared_ptr` / `vector` / `string` -ն ինքն իրեն են կառավարում հիշողությունը:

4.3 Container-ներ (vector, string)

```
std::vector<int> v(100);    // heap-ը ինքն է կառավարում, ոչ մի leak  
std::string s = "hello";   // նույնը
```

5. Ինչպես հայտնաբերել (գործիքներ)

Valgrind (Linux):	valgrind --leak-check=full ./program
AddressSanitizer:	g++ -fsanitize=address ... (ASan)
LeakSanitizer:	-fsanitize=leak
Visual Studio:	CRT Debug Heap / _CrtDumpMemoryLeaks()

Օրինակ ASan-ով.

```
g++ -fsanitize=address -g leak.cpp -o leak
./leak
# -> "detected memory leaks", stack trace` որտեղ է new-ը
```

6. Հարցազրույցի հարցեր և պատասխաններ

Memory leak ինչ է:

Heap հիշողություն, որ վերցվել է բայց չի ազատվել, և այլևս հասցե չկա; մնում է զբաղված մինչև ծրագրի ավարտը:

Երբ է լինում:

delete-ի մոռացում, early return/exception, pointer overwrite, delete vs delete[] սխալ:

Ինչպես խուսափել:

RAII + smart pointers (unique_ptr/shared_ptr), STL container-ներ (vector/string), ամեն new-ի համար delete:

Ինչպես հայտնաբերել:

Valgrind, AddressSanitizer/LeakSanitizer, CRT debug heap:

delete vs delete[]:

new -> delete; new[] -> delete[]. Սխալ գույգը -> undefined behavior / leak:

Modern C++-ում ինչպես ես խուսափում leak-ից:

make_unique/make_shared, ոչ մի raw new/delete; container-ներ: Rule of Zero:

Ամփոփում (Cheat Sheet)

MEMORY LEAK = heap հիշողություն վերցված, չի ազատվել, հասցեն կորել

ՊԱՏՃԱՌՆԵՐ:

delete-ի մոռացում | early return/exception | pointer overwrite | delete vs delete[]

ԽՈՒՍԱՓԵԼ:

RAII + smart pointers (make_unique/make_shared)

STL container-ներ (vector/string)

ամեն new -> delete (+ nullptr)

ԳՈՐԾԻՔՆԵՐ:

valgrind --leak-check=full | -fsanitize=address (ASan) | -fsanitize=leak

ԿԱԼՈՆ: modern C++ -> ոչ raw new/delete -> ոչ leak (Rule of Zero)